

Analyzing Throughput of GPUs Exploiting Within-Die Core-to-Core Frequency Variation

Jungseob Lee, Paritosh Ajgaonkar, Nam Sung Kim

Apr 12, 2011

**Department of Electrical and Computer Engineering
University of Wisconsin - Madison**

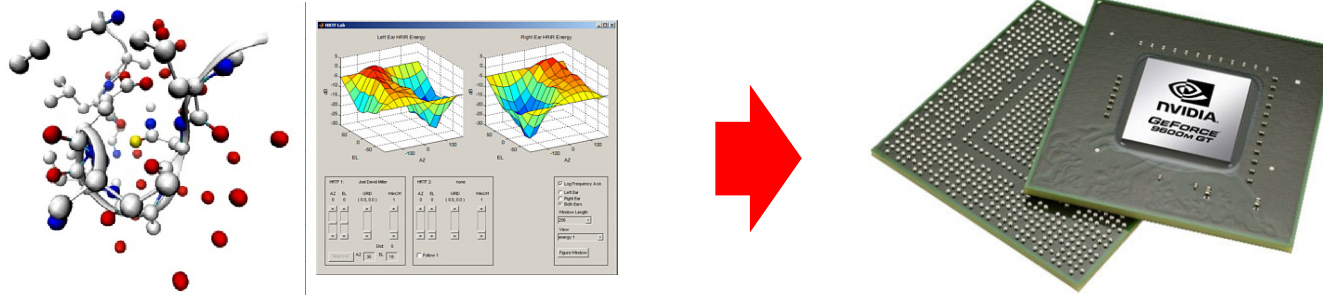


Outline

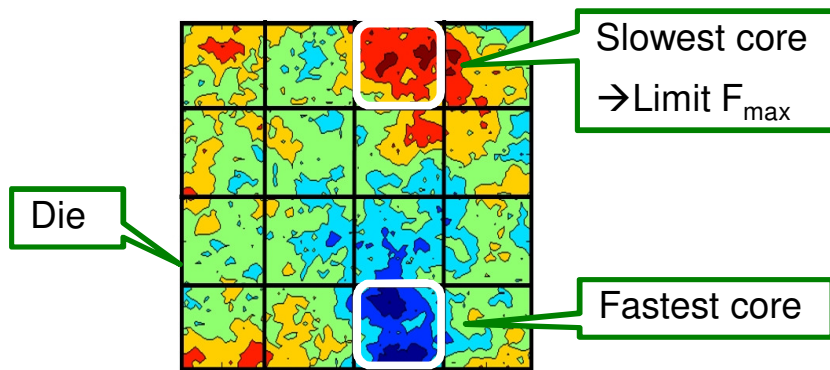
- **Introduction**
- GPU architecture and impact of WID variations on GPUs
- Throughput improvement techniques
 - ⊙ Allowing per-SM clocking (PSMC)
 - ⊙ Disabling the slowest SMs (DSSM)
- Impact of main memory latency and bandwidth on throughput improvement
- Conclusion

Introduction

- Improve throughput of GPU applications



GPUs can provide high throughput for general-purpose and data-intensive applications.



Increasing WID Core-to-Core frequency variations affect $F_{\max}$ of GPUs.

PSMC & DSSM

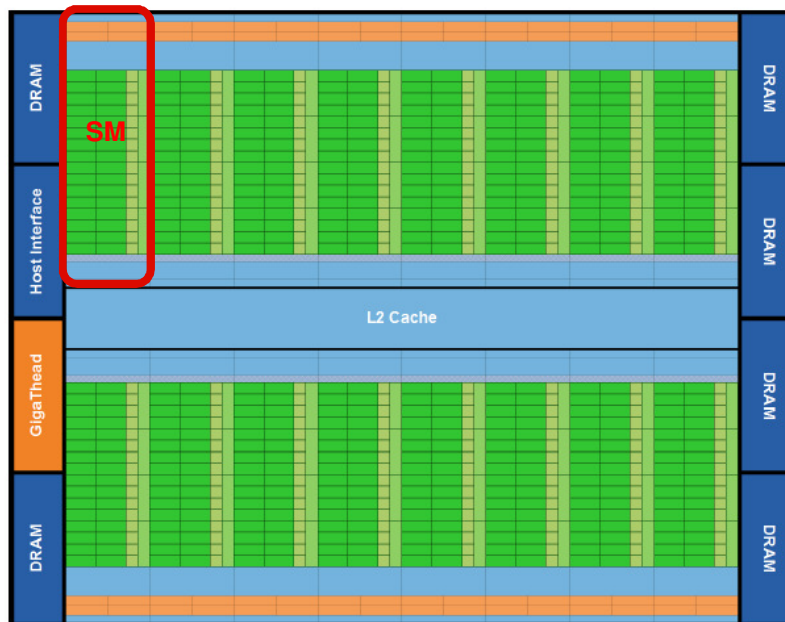
Two techniques for mitigating the negative impact of WID C2C frequency variations on throughput of GPUs

Outline

- Introduction
- **GPU architecture and impact of WID variations on GPUs**
- Throughput improvement techniques
 - ⦿ Allowing per-SM clocking (PSMC)
 - ⦿ Disabling the slowest SMs (DSSM)
- Impact of main memory latency and bandwidth on throughput improvement
- Conclusion

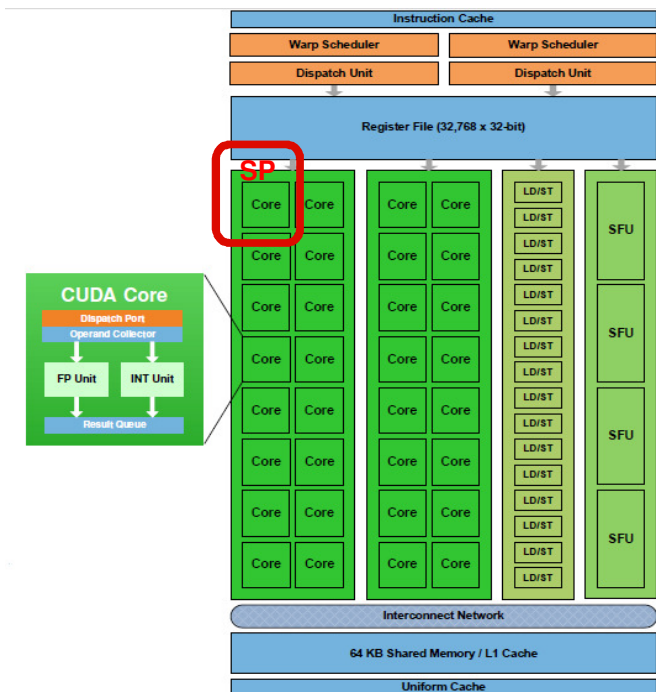
GPU architecture

- GPU architecture:
 - ◉ Streaming Multiprocessor (SM)s, off-chip DRAM, and on-chip interconnect network
 - ◉ Each SM: 1) 8 to 32 streaming processors (SPs), 2) an instruction scheduler, 3) instruction cache, 4) register files, 5) special function units (SFU), and 6) shared memory/cache



Fermi's 16 SM are positioned around a common L2 cache. Each SM is a vertical

Fermi Architecture [1]



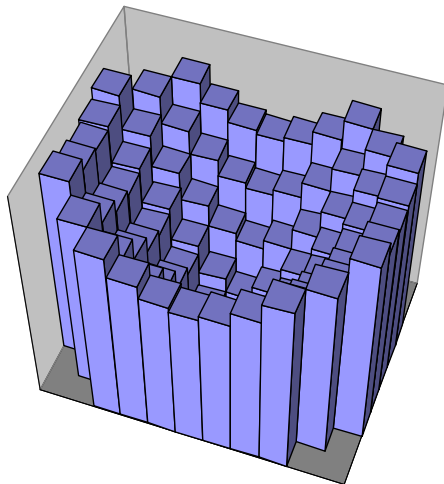
Streaming Multiprocessor (SM) [1]

[1] http://www.nvidia.com/content/PDF/fermi_white_papers/NVIDIA_Fermi_Compute_Architecture_Whitepaper.pdf

D2D & WID variations

- Die to Die (D2D) variations
 - ⊙ Affect all transistors on a die identically
- Within-Die (WID) variations
 - ⊙ Different transistor characteristics within a single die.
 - ⊙ With technology scaling (i.e., more cores per a die)
 - Considerable C2C F_{MAX} variation due to spatially correlated WID variations

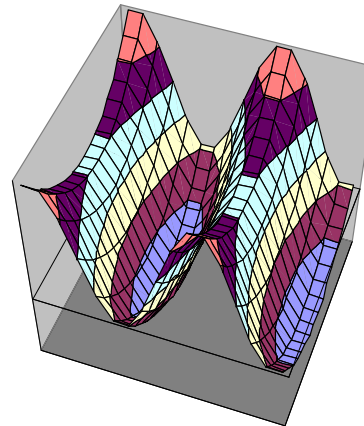
Die-to-Die (D2D) Variations



Wafer Scale

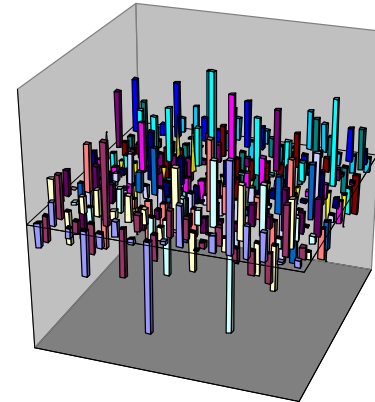
Within-Die (WID) Variations

Systematic



Die Scale

Random

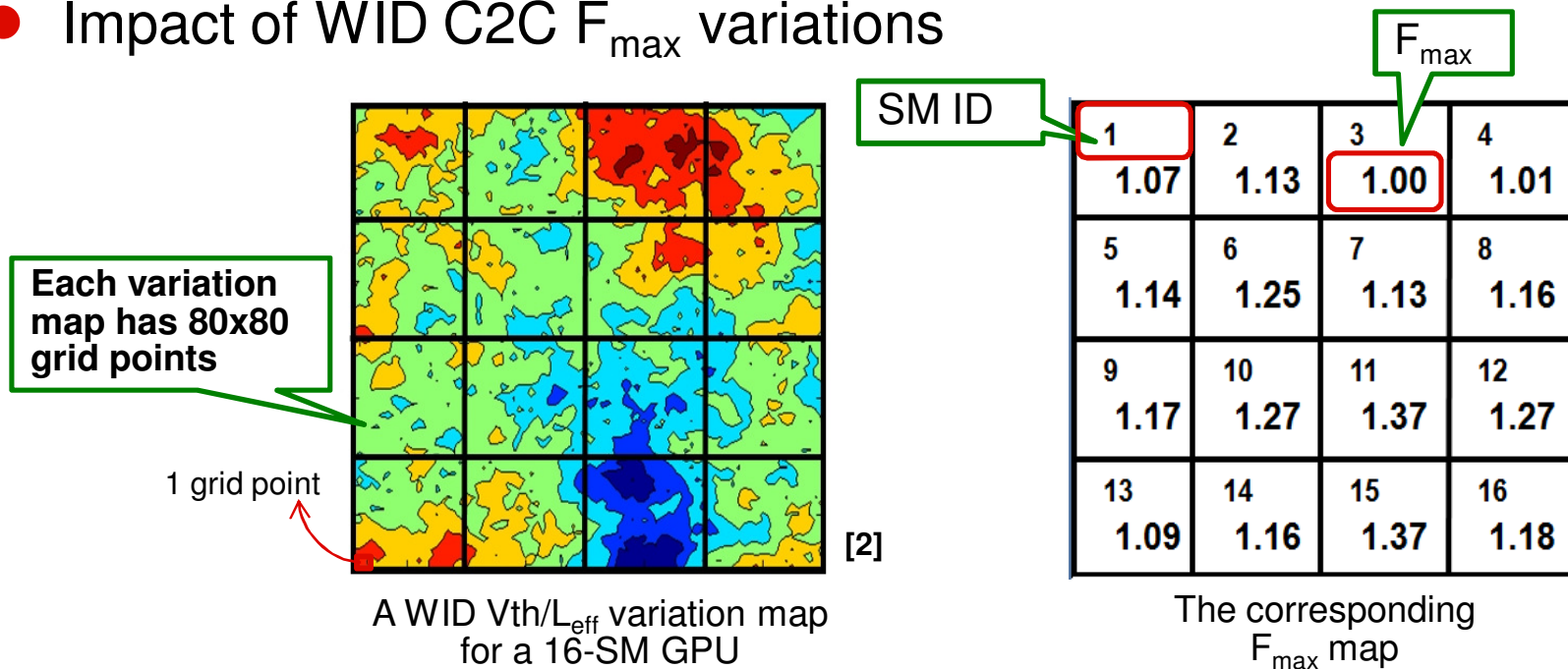


Feature Scale

Courtesy: K. Bowman from Intel

WID C2C F_{MAX} Variations

- Impact of WID C2C F_{max} variations



- C2C frequency variation affects GPU's F_{MAX}
 - F_{max} is limited by the slowest core in a GPU designed to operate all SMs at the same frequency (i.e, per-chip clocking).
- More SMs in a die → SM-to-SM frequency variation increases
- Power inefficiency → faster SMs consume more leakage power

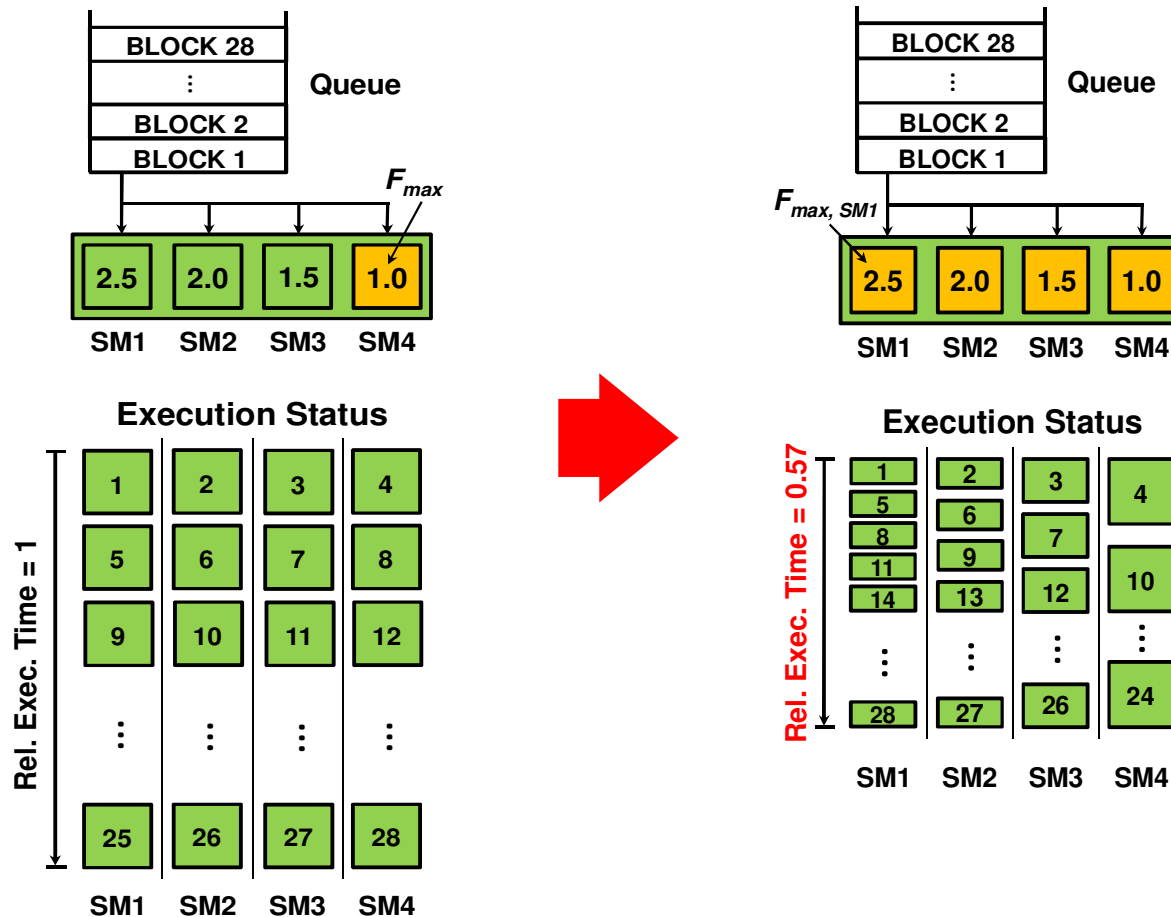
[2] S. Herbert et al., "Characterizing chip-multiprocessor variability-tolerance," in Proc. IEEE DAC, 2008.

Outline

- Introduction
- GPU architecture and impact of WID variations on GPUs
- **Throughput improvement techniques**
 - ⦿ **Allowing per-SM clocking (PSMC)**
 - ⦿ Disabling the slowest SMs (DSSM)
- Impact of main memory latency and bandwidth on throughput improvement
- Conclusion

Per-SM clocking (PSMC)

- Each SM executing independent thread blocks
 - ⦿ Enabling PSMC efficiently for GPUs w/ per-SM PLL
 - Many SP-to-SP communications through a shared memory in an SM



GPGPU-Sim config. and benchmarks

⦿ GPGPU-Sim parameters

Number of Core (SM)s	16 / 32 / 64	Memory Channels	4 / 8 / 8
Core (SM) Frequency (GHz)	1.688 / 1.476 / 1.401	Memory Frequency (GHz)	1.100 / 1.242 / 1.848
Interconnection Frequency (GHz)	0.85 / 1.00 / 1.50	Memory bandwidth (GB/s)	70.4 / 159 / 236.5
Warp Size	32	Bandwidth / Memory module	4 (Bytes/Cycle)
SIMD Pipeline Width	8	Memory Controller	FR-FCFS
Number of Threads / Core	1024	Branch Divergence Method	Immediate Post Dominator
Number of CTAs / Core	8	Warp Scheduling Policy	Round Robin
Number of Regs / Core	16384	Constant Cache Size / Core	8 KB
Shared Memory / Core	16 KB	Texture Cache Size / Core	8 KB

⦿ 12 CUDA benchmarks [3, 4]

AES encryption (AES)	Black-scholes (BLK)	gpuDG (DG)
3D Laplace Solver (LPS)	Ray Tracing (RAY)	StoreGPU (STO)
Breadth-First Search (BFS)	LIBOR Monte Carlo (LIB)	MUMmerGPU (MUM)
Neural Network (NN)	Image Denoising (IMG)	Sum of Absolute Differences (SAD)

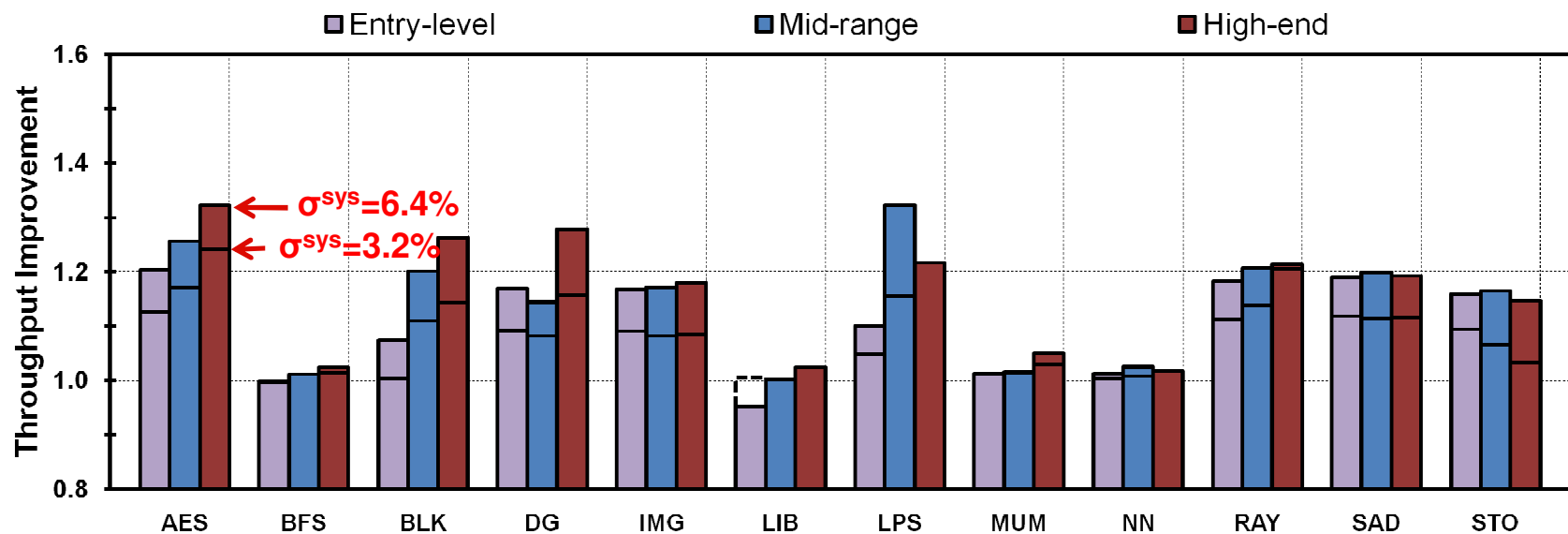
[3] A. Bakhoda., "Analyzing cuda workloads using a detailed GPU simulator," in Proc. ISPASS, 2009.

[4] "ERCBench, A Benchmark Suite for Embedded and Reconfigurable Computing," <http://ercbench.ece.wisc.edu/index.php>

Per-SM clocking (PSMC)

- Theoretical throughput improvement:

- $Speedup = \left(\sum_{i=1}^N (F_{max,i} / F_{max,slowest}) \right) / N$



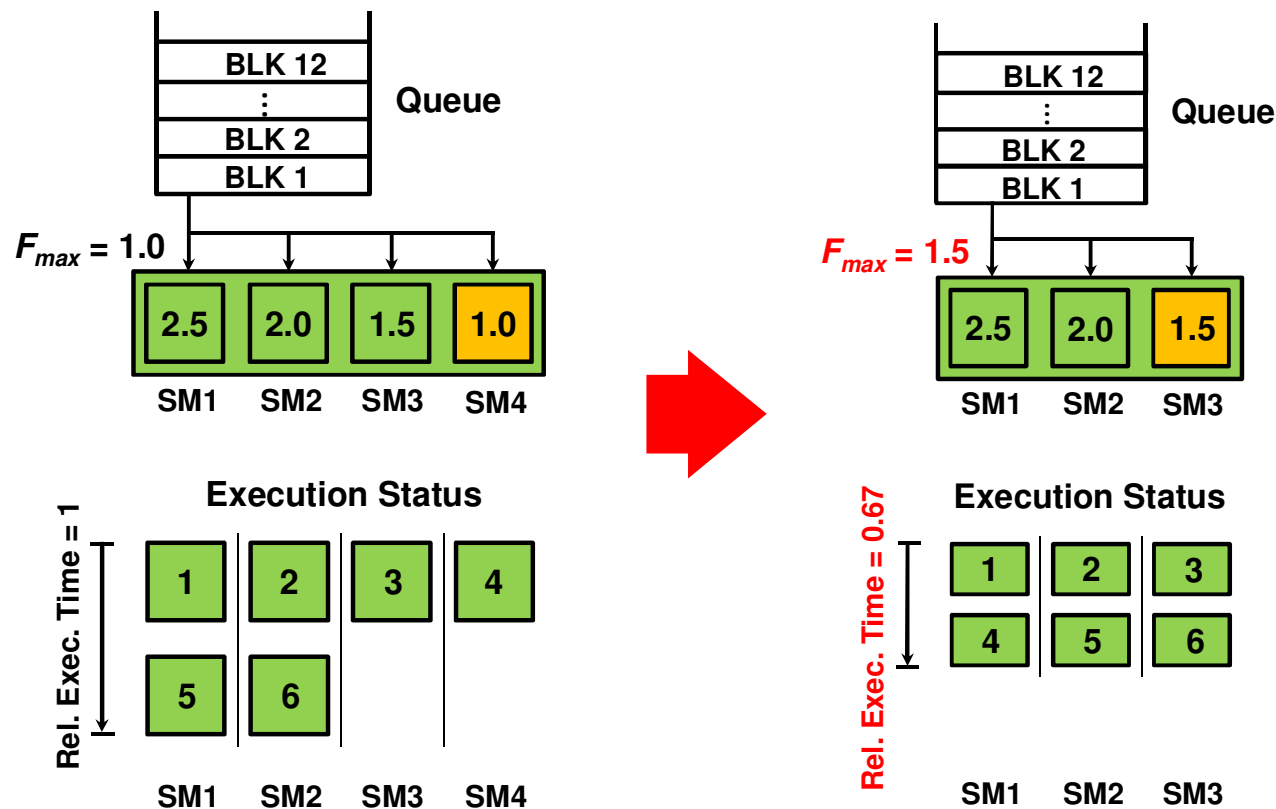
- 10%, 14%, and 16% higher throughput for entry-level, mid-range, and high-end GPUs on average.

Outline

- Introduction
- GPU architecture and impact of WID variations on GPUs
- **Throughput improvement techniques**
 - ⦿ Allowing per-SM clocking (PSMC)
 - ⦿ **Disabling the slowest SMs (DSSM)**
- Impact of main memory latency and bandwidth on throughput improvement
- Conclusion

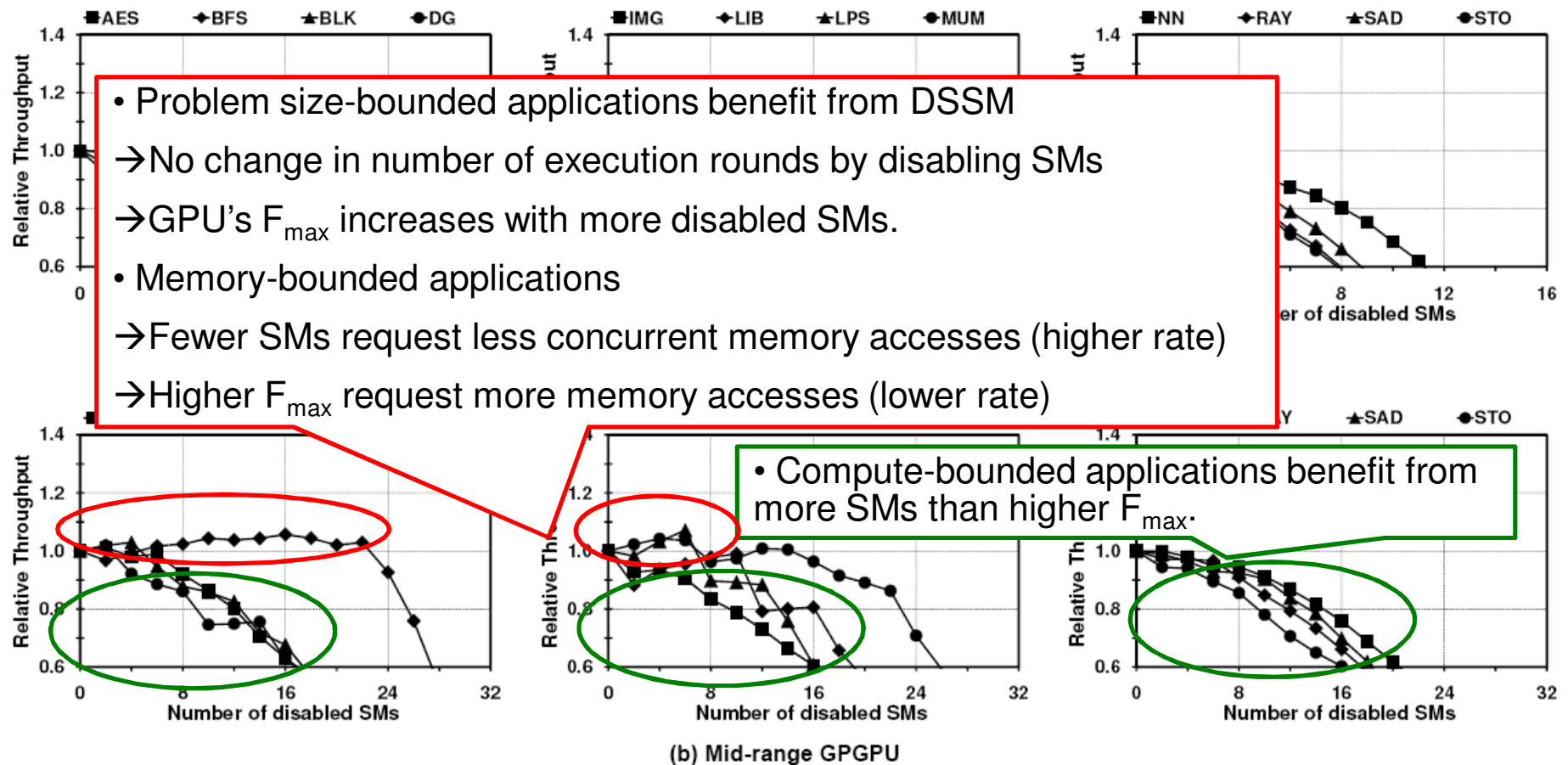
Disabling the slowest SM (DSSM)

- Problem size-bounded applications
 - small problem size relative to the number of SMs
 - throughput does not increase with more available SMs
- Disabling the slowest SM(s) $\rightarrow$ higher $F_{\max}$ for the GPU



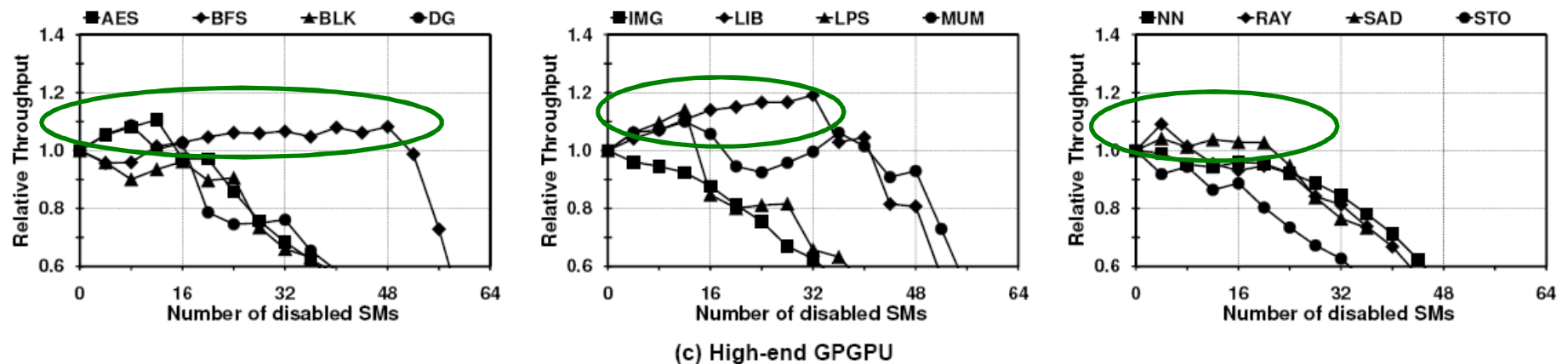
Disabling the slowest SM (DSSM)

- The slowest SMs are disabled one by one
- Relative throughput of 12 applications for entry-level and mid-range GPU



Disabling the slowest SM (DSSM)

- The slowest SMs are disabled one by one
- Relative throughput of 12 applications for high-end GPU



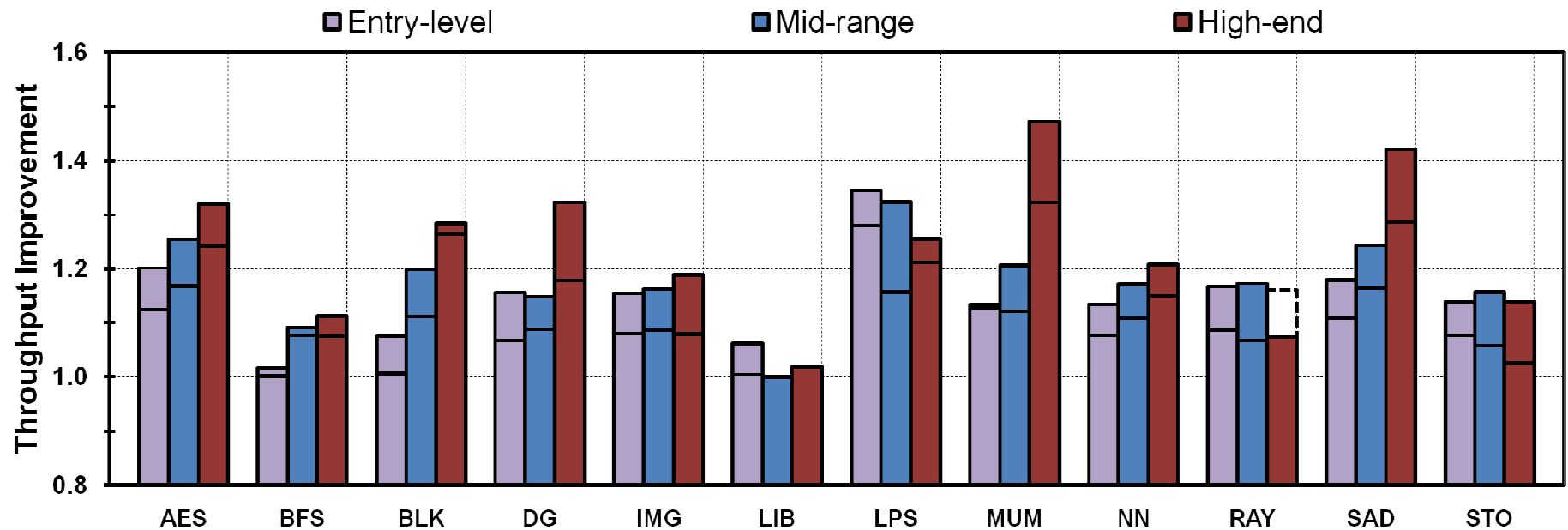
- If appropriate number of the slowest SMs are disabled (i.e., 2 to 6 out of 32 SMs and 4 to 32 out of 64 SMs)
 - 1%~7% and 4%~19% throughput improvement for certain applications

Outline

- Introduction
- GPU architecture and impact of WID variations on GPUs
- Throughput improvement techniques
 - ⦿ Allowing per-SM clocking (PSMC)
 - ⦿ Disabling the slowest SMs (DSSM)
- **Impact of main memory latency and bandwidth on throughput improvement**
- Conclusion

Per-SM clocking (PSMC)

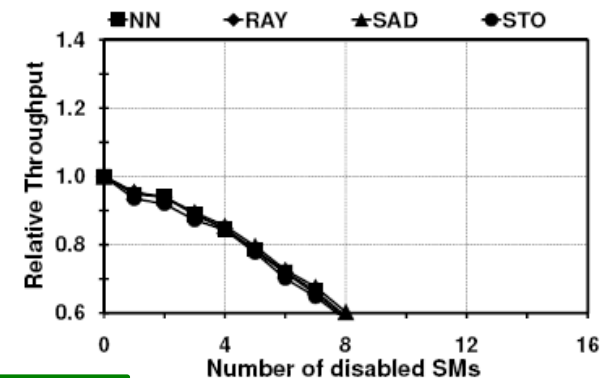
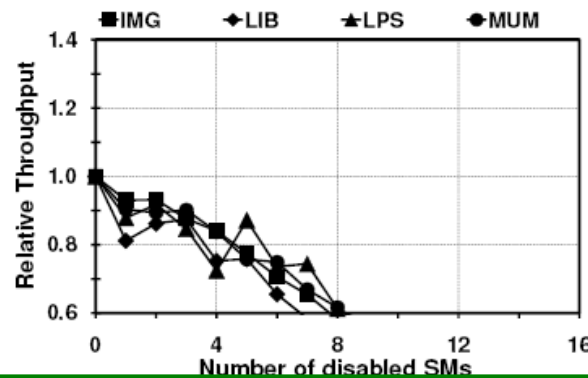
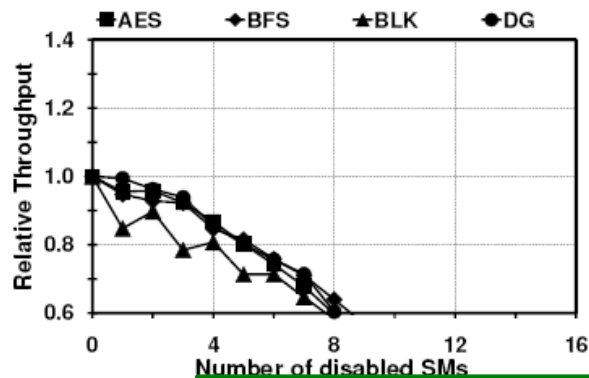
- Emerging memory technology
 - 32% lower latency and 6 times higher bandwidth
- Relative throughput improvement of applications adopting the PSMC scheme



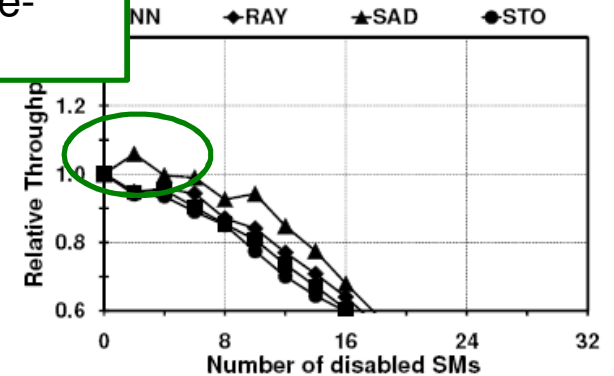
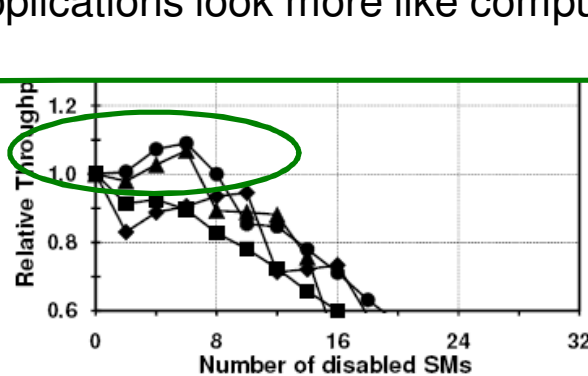
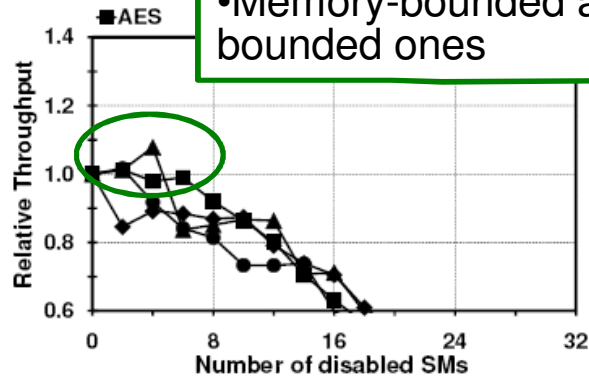
- 15%, 18%, and 24% higher throughput than the baselines for entry-level, mid-range, and high-end GPUs on average

Disabling the slowest SM (DSSM)

- The slowest SMs are disabled one by one
- Relative throughput of 12 applications for entry-level and mid-range GPUs



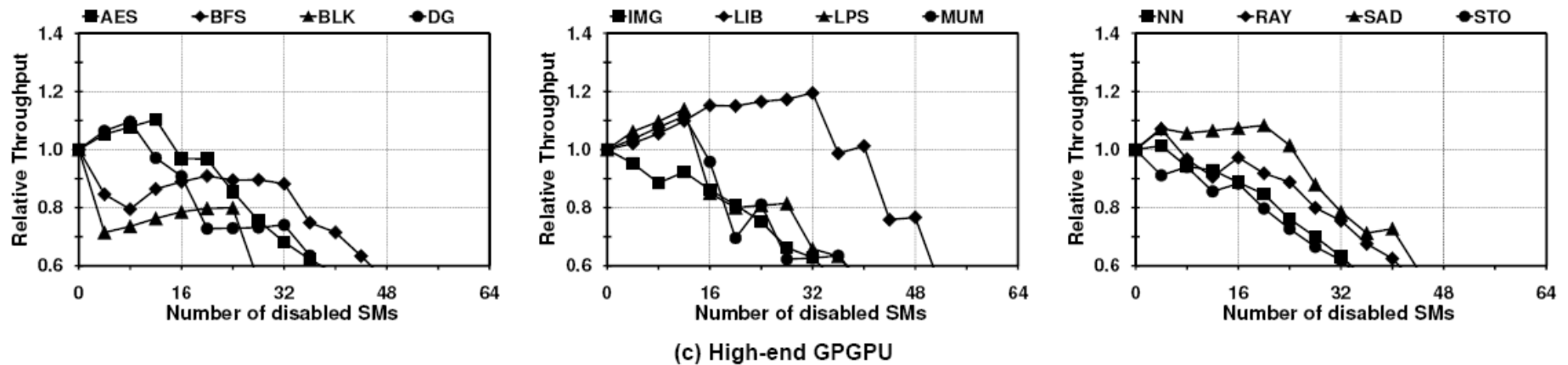
- Problem size bounded applications still benefit from DSSM
- Memory-bounded applications look more like compute-bounded ones



(b) Mid-range GPGPU

Disabling the slowest SM (DSSM)

- The slowest SMs are disabled one by one
- Relative throughput of 12 applications for high-end GPU



- If appropriate number of the slowest SMs are disabled for high-end GPU
 - 7%~20% throughput improvement for certain applications

Conclusion

- Two throughput improvement techniques to exploit WID SM-to-SM frequency variations in GPUs.
 - ⊙ Allowing each SM to operate at its own $F_{\max}$
 - ⊙ Disabling the slowest SMs
- PSMC
 - ⊙ 10%~16% throughput improvement of applications on average
- DSSM
 - ⊙ up to 19% throughput improvement of applications
- Impact of main memory latency and bandwidth on throughput
 - ⊙ Emerging memory technology
 - lower latency and higher bandwidth
 - ⊙ Throughput improvement by up to 24% and 20%